

基于 8051F310 单片机的垃圾分类系统设计

成绩

指导教师：罗志祥

专业班级：光电信息科学与工程 202409

姓名：陈恪瑾

学号：U202413925

联系电话：18571851291

1 任务要求



图 1: 工作场景：小区垃圾回收系统

1.1 实验目的

- 熟悉 C8051F310 单片机实验平台的 I/O 口、矩阵键盘、数码管、流水灯和蜂鸣器等外设资源的配置方法；
- 掌握 4 位数码管动态扫描显示、4×4 矩阵键盘扫描与消抖、74HCT164 串行驱动流水灯等常用接口程序设计方法；
- 理解定时器中断和外部中断的工作机制，能够利用 Timer0 构造 1ms 系统节拍，并在主循环状态机中完成倒计时、闪烁和报警等并发任务；

- 通过完整汇编程序的编写、构建与调试，提升对模块化程序结构、共享 RAM 状态变量和嵌入式系统故障定位方法的掌握程度。

1.2 实验内容

本设计基于 C8051F310EVM 实验平台，采用 MCS-51 汇编语言实现小区智能垃圾分类回收系统。系统以四位数码管显示四类垃圾桶剩余容量，以 F1-F4 分类按键完成开盖与投递，以 Timer0 和外部中断分别完成精确定时及提前关盖控制。具体要求如下。

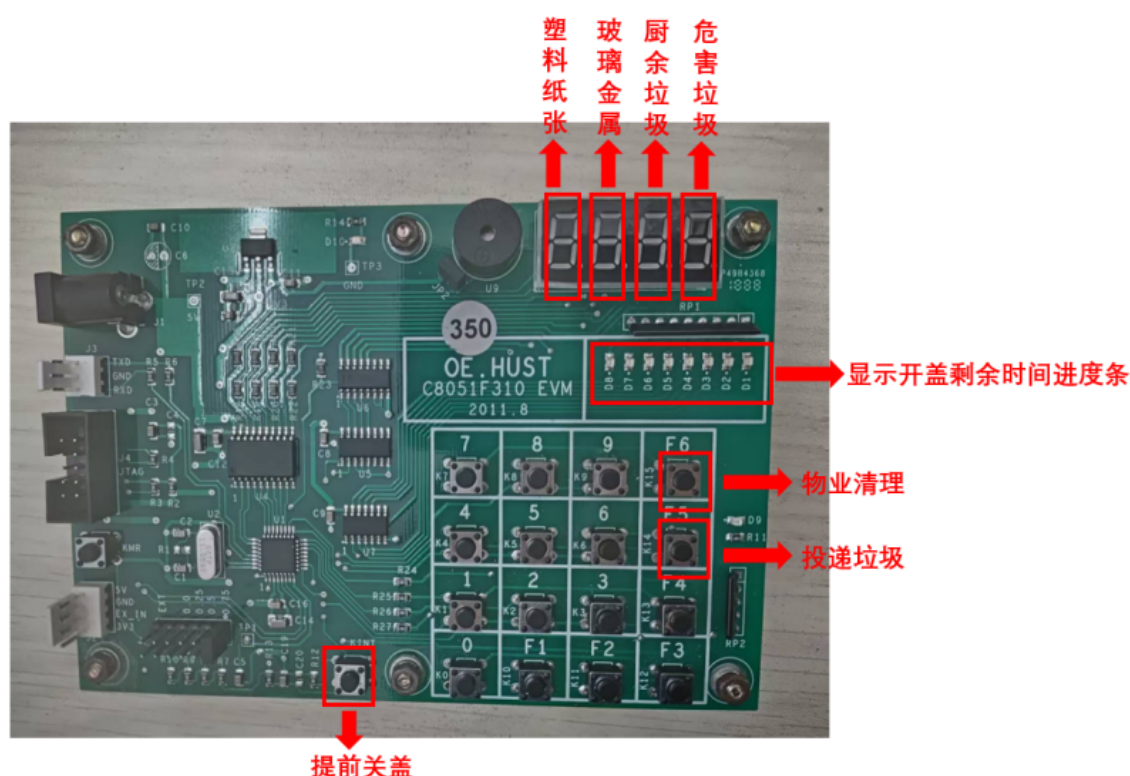


图 2: 开发板示意图

基础部分（满分 85）

- 系统将生活垃圾分为“塑料纸张”、“玻璃金属”、“厨余垃圾”和“危害垃圾”四类。上电初始化后，程序随机生成 4 个 0 ~ 9 的整数，分别作为四个垃圾桶的剩余可投放容量，并在四位数码管上同时显示。
- 用户按下 K10-K13 (F1-F4) 中的某一分类键选择对应垃圾桶。若该桶剩余容量为 0，则认为垃圾桶已满，蜂鸣器鸣响约 1s，对应数码管显示字母 F (FULL)，提示结束后恢复原容量显示，桶盖不开启。

- 若所选垃圾桶仍有剩余容量，则系统打开对应桶盖，D9 指示灯点亮，开盖时间设定为 8s；开盖期间，对应数码管以 5Hz 频率闪烁，显示该桶当前剩余容量。
- 在桶盖打开的 8s 内，用户再次按下同一个分类键表示投入 1 袋垃圾，系统将该桶容量减 1；该操作可重复执行，直至投放完成或容量减至 0。
- 桶盖计时由 Timer0 中断完成。程序以 1ms 为系统节拍维护 16 位倒计时变量，要求开盖 8s 后自动关闭桶盖，D9 指示灯熄灭，选中数码管停止闪烁。

1.3 提高部分（申优选做，85 分以上）

在基本功能基础上，当前程序进一步实现了以下扩展功能：

- 任一垃圾桶被选择后，四位数码管仍同时刷新显示全部垃圾桶状态；其中被选中的桶位通过 5Hz 闪烁突出显示，未选中的桶位保持正常容量显示。
- 桶盖开启期间，流水灯 D1–D8 作为 8s 倒计时进度条显示剩余开盖时间：开盖初始全亮，随后按秒级阈值逐格熄灭，倒计时结束后全部熄灭。
- 开盖期间支持 K1–K9 数字键批量投放。按下数字键 n 表示一次投入 n 袋垃圾；若当前容量足够，则容量立即减 n ；若投入袋数超过剩余容量，则蜂鸣器鸣响约 1s，对应数码管显示 E (ERROR)，并关闭桶盖拒绝本次投放。
- 若用户在 8s 内已完成投放，可按 Kint 外部中断键提前关盖。外部中断服务程序立即清除开盖状态和倒计时变量，使 D9 与 D1–D8 熄灭，系统返回待机状态。
- 增加物业维护功能：按下 K15 (F6) 后，四个垃圾桶容量统一恢复为 9，同时关闭当前开盖状态并鸣响蜂鸣器作为确认。
- 增加计算器个性化功能：按下 K14 (F5) 进入计算器模式，K0–K9 输入数字，K10–K13 分别表示加、减、乘、除，K15 (F6) 执行等号运算，K14 (F5) 用于退格或返回上一步，Kint 用于退出计算器模式并恢复垃圾分类系统。

2 设计思路

本系统基于 C8051F310 单片机实验平台，采用纯 MCS-51 汇编语言实现。整体程序按照“硬件初始化 + 主循环状态机 + Timer0 周期中断 + INT0 外部中断 + 独立外

设驱动模块”的结构组织，将垃圾分类业务、显示刷新、键盘扫描、流水灯输出和计算器扩展功能分离到不同源文件中，便于调试与后续维护。

系统的核心思想是：主循环只负责刷新显示、扫描键盘并根据当前状态执行一次性业务动作；所有需要精确定时的任务交由 Timer0 中断维护；所有需要快速响应的提前关盖动作交由外部中断完成。程序通过片内 RAM 中的状态变量保存四个垃圾桶容量、当前开盖桶号、提示覆盖桶号、错误覆盖桶号、倒计时计数值和计算器模式状态，从而避免长时间阻塞延时，使数码管显示、蜂鸣器报警和按键响应能够同时进行。

整体设计逻辑如下：

1. **系统初始化与随机容量生成模块**：程序复位后首先关闭看门狗，配置 24.5MHz 内部振荡器、端口输出模式、Timer0 工作方式和 INT0 下降沿触发方式。随后初始化片内 RAM 中的容量、键盘、显示、倒计时、报警和计算器状态变量。系统利用 Timer0 自由运行值与端口输入状态组合形成伪随机种子，经乘法、加法和除 10 取余运算后生成 CAP0~CAP3，作为四个垃圾桶的初始剩余容量。
2. **主循环轮询与业务状态机模块**：主循环首先调用流水灯更新函数，再连续刷新四位数码管以保持稳定显示，然后扫描 4×4 矩阵键盘。键盘返回值通过 KEYLAST 进行边沿检测，避免长按造成重复触发。在普通垃圾分类模式下，K10~K13 被解释为四类垃圾桶选择键，K1~K9 被解释为批量投放袋数，K14 进入计算器模式，K15 执行物业清空。主循环根据 LIDOPEN、SELBIN 和容量值判断当前应开盖、投放、拒投还是忽略无效按键。
3. **开盖与容量扣减模块**：当用户按下某一分类键且该桶容量不为 0 时，程序设置 LIDOPEN=1，记录当前桶号到 SELBIN，装入 8000ms 倒计时初值，并点亮 D9 表示桶盖打开。开盖期间，再次按下同一分类键将投放 1 袋垃圾；按下 K1~K9 则调用同一容量处理过程一次性投放指定袋数。若当前容量足够，程序直接扣减对应 CAP 变量；若容量不足，则设置 ERBIN 显示 E，启动 1s 蜂鸣提示，并立即关闭桶盖。
4. **Timer0 多任务系统节拍模块**：Timer0 工作在 16 位定时方式，按 1ms 周期重装初值。中断服务程序同时维护三类时间任务：一是递减 LIDHI:LIDLO 组成的 8s 开盖倒计时，计数归零后自动清除开盖状态并熄灭 D9；二是每 100ms 翻转一次 BLKFLG，为选中数码管提供 5Hz 闪烁节拍；三是递减 HINTHI:HINTLO 组成的 1s 提示倒计时，提示结束后关闭蜂鸣器并清除 F/E 覆盖显示。
5. **外部中断快速关盖模块**：INT0 与 Kint 按键相连，采用下降沿触发。普通模式下，若桶盖处于打开状态，外部中断服务程序立即清除 LIDOPEN、SELBIN 和倒计时变

量,并熄灭 D9,实现不依赖主循环扫描周期的提前关盖。若系统处于计算器模式,Kint 则被复用为退出键,用于清除计算器状态并回到垃圾分类模式。

6. **动态数码管显示与覆盖优先级模块:** 四位数码管采用分时动态扫描方式显示。普通模式下,各位分别显示 CAPO~CAP3;显示段码由 GET_SEG 统一决定。其覆盖优先级为:若当前位等于 ERRBIN 则显示 E,若等于 FULLBIN 则显示 F,若等于开盖桶号且 BLKFLG 处于熄灭相位则显示空白,否则显示正常容量数字。通过这种掩码覆盖方法,程序无需改变容量缓存即可完成错误、满桶和闪烁提示。
7. **流水灯倒计时进度条模块:** D1~D8 由 74HCT164 串行移位寄存器驱动,程序使用 LEDREG 保存当前输出掩码,使用 LASTBAR 保存上一次输出值。主循环根据 8000ms 倒计时剩余值映射出 0~8 个亮灯等级,只有掩码变化时才调用移位输出函数,从而减少无效刷新。桶盖关闭时流水灯强制全灭。
8. **提示报警与物业清空模块:** 当待机状态下选择容量为 0 的垃圾桶时,程序设置 FULLBIN 并启动蜂鸣器,使对应数码管显示 F 约 1s。按下 K15 (F6) 时,物业清空函数将四个容量变量统一恢复为 9,关闭当前开盖状态,清空流水灯,并鸣响蜂鸣器作为确认反馈。
9. **计算器个性化扩展模块:** 为体现功能扩展能力,程序加入独立计算器模式。按 K14 (F5) 进入计算器后,垃圾分类相关状态被暂停并清空提示输出,数码管转为显示计算器缓存 CALCD0~CALCD3。K0~K9 用于输入最多两位的操作数,K10~K13 分别表示加、减、乘、除,K15 执行等号计算,K14 用于退格或返回。计算完成后,结果通过同一套数码管动态扫描函数显示,Kint 可随时退出并恢复正常垃圾分类模式。

3 资源分配

3.1 I/O 口硬件资源分配

I/O 引脚	方向	功能定义	有效电平
P0.0	输出	D9 桶盖开启指示灯控制	低电平开启
P0.6, P0.7	输出	74HC139 数码管位选地址线	低电平脉冲配合翻译
P1.0~P1.7	输出	数码管段码输出 (A~DP)	低电平驱动显示
P2.0~P2.3	输出	键盘行线扫描	低电平使能
P2.4~P2.7	输入	键盘列线检测	低电平有效
P3.2 (INT0)	输入	外部中断 0 触发键 (提前关盖)	下降沿触发
P3.3	输出	74HC164 串行数据输出 (DAT)	高低电平动态输出
P3.4	输出	74HC164 移位时钟输出 (CLK)	上升沿脉冲触发

3.2 寄存器资源分配

寄存器	功能定义
SP	堆栈指针, 初始化为 5FH
A, B	数据中转、数学乘除运算 (容量随机化计算)、状态及延时比较等
DPTR	映射表与段码查表基址指针
R0	SHIFT_LED_BAR 流水灯 8 位移位的循环控制计数器
R5, R6, R7	数码管单字符延时 (DELAY_DIG) 与按键消抖的内层与外层计数器

3.3 片内 RAM 资源分配

存储单元	功能定义
30H~33H	CAP0~CAP3 暂存四个分类垃圾桶的剩余容量 (初始化赋 0~9)
34H, 35H	KEYNOW, KEYLAST 用于保存及判断双边沿防连按防抖的键号
39H, 3BH	LEDREG, LASTBAR 存放将要推入与前次推入 74HC164 的掩码态
42H, 43H	LIDLO, LIDHI 组成双字节作为 8 秒 (8000ms) 主业务关盖倒计时器
44H, 45H	BLKDIV, BLKFLG 用于累加计算 100ms 阈值与倒转产生的 5Hz 发光控制
46H, 47H	HINTLO, HINTHI 组成双字节作为 1 秒 “F” (FULL) 报错的阻塞倒数
4DH, 4EH	LIDOPEN, SELBIN 保存当前盖子开启标志极性态与正在受理的舱位
4FH	FULLBIN 保存当前容量为空无法继续时被点名弹出满载报错显示的舱号

4 流程图

4.1 主程序状态轮询架构流程图

4.2 定时器/外部中断并行处理流程图

4.3 动态掩码刷新流图

5 源代码（含文件头说明、语句行注释）

6 程序测试方法与结果

本节在 Proteus 仿真环境下对智能垃圾分类投放系统的各项功能进行测试，重点验证随机容量生成、开盖逻辑、容量递减、满载提示、倒计时流水灯进度条及外部中断提前关盖等功能的正确性。

6.1 测试方法

1. **初始化容量生成测试**：系统上电或复位后，观察四位数码管是否随机生成 0 ~ 9 的数字，流水灯 D1~D8 及指示灯 D9 处于熄灭状态。
2. **正常投放开盖及频闪测试**：当对应类别存在剩余容量时，按下键盘对应组键，检查 D9 开启、且该对应数码管是否以 5Hz 频率闪烁、流水灯 D1~D8 是否全亮。
3. **投放容量递减测试**：在开盖倒计时 8 秒期间，再次按下同一按键，观察闪烁中的数码管对应的容量数字是否递减 1。
4. **硬件倒计时及打断测试**：观察流水灯是否随着 8 秒时间流逝逐格熄灭。开盖期间点击外部中断按键，观察盖子和流水灯是否即刻关闭。
5. **满载报错全显闪现测试**：针对容量为 0 的垃圾分类，在待机时按下按键，观察数码管对应位是否短暂显示字母 F 并持续 1 秒后恢复。

6.2 测试结果

（1）上电随机容量显示测试

操作：系统上电运行。

预期：四位数码管显示 4 位独立生成的随机数（例如“2 5 8 1”），指示灯全灭。

结果：符合预期。

（2）投放开盖与频闪进度条激活测试

操作：按下一个有余量的分类对应按键。

预期：D9 灯亮模拟开盖，选中的数码管数字开始闪烁，D1~D8 流水灯全亮。

结果：符合预期。

（3）舱室容量动态扣减测试

操作：在上述开盖期间，再次按下该分类对应的按键。

预期：正在闪烁的数码管容量减 1，其它状态及计数不变。

结果：符合预期。

(4) 进度条流失与提前中止中断测试

操作：不操作按键等待进度条自然熄灭；随后再次开盖，途中按下外部中断按钮。

预期：进度条随秒数等比递减，直到熄灭（自动关盖）；有外部中断触发时，倒数全盘清空并立刻关盖息屏。

结果：符合预期。

(5) 满载报错提示拦截测试

操作：将某分类垃圾桶容量扣减为 0，桶盖关闭待机。此时在此分类下尝试二次投递按键。

预期：系统拒绝开盖，流水灯不亮，在对应的数码管暂留“F”（FULL）警告，约 1 秒后自动消失并恢复原本数字 0 的空闲显示。

结果：符合预期。

综上所述，程序完整且流畅地实现了在多按键输入场景下的各类垃圾存放分配功能。单片机依靠其内外部中断系统与状态机轮询完美承接了题目所要求的进度条和闪烁控制等所有加分冲优任务，逻辑判定响应均无短频或明显时延。

7 实验问题分析与对策

1. Proteus 仿真软件存在 bug:

- **原因分析：**Proteus 仿真软件存在 bug，无论如何修改代码，Proteus 对于按键的输入引脚电平计算有误，相同的代码和工程文件在另一台电脑上即可正常运行。
- **对策：**借助同学的电脑进行，完成实验。

2. 数码管显示字形与预期不符:

- **原因分析：**段码与电路接线顺序不匹配，数字显示不正常。
- **对策：**对照原理图逐一确认 P1.0~P1.7 分别对应 DP、G、F、E、D、C、B、A 引脚，重新计算共阴极段码：S=0B6H、O=0FCH、E=09EH、I=060H，验证后问题解决。

3. 按键扫描的逻辑不正确:

- **原因分析:** 理论上按键在扫描时应该将 P2.0~P2.3 依次置低, 读取 P2.4~P2.7 的状态来判断按键, 但实际代码中将电平逻辑反了, 而硬件连接不支持另一种电平逻辑, 导致按键状态无法正确识别。
- **对策:** 修改按键扫描代码, 将 P2.0~P2.3 置低时读取 P2.4~P2.7 的状态进行判断, 确保按键输入能够正确映射到显示缓存的更新。

本人承诺: 本报告内容真实, 无伪造数据, 无抄袭他人成果。本人完全了解学校相关规定, 如若违反, 愿意承担其后果。

签字:

陈恪瑾

2026 年 4 月 26 日